

REST API Demo

Siddharth Ananth and Robin Liu

Self Introduction - Siddharth Ananth

- Originally from Mumbai, India
- BS in Computer Science from Johns Hopkins University
- Currently pursuing MS in Computer Science
- 4 Software Engineering Internships (focused mainly on C++ and Python) + 3 to 4 years in research experience
- Hobbies: improv comedy, bollywood movies, boxing (no sparring)
- [LinkedIn](#)



REST API Basics

REST (Representational State Transfer) is a set of rules for building APIs that allow programs to communicate over the internet.

Carries out **Client-Server Communication**:

- The **client** (a web browser, mobile app, etc.) sends a request.
- The **server** processes the request and sends back a response.



Key Features:

- **Stateless:** Each request from the client must contain all the necessary information because the server doesn't remember previous requests
- **Standard Methods:** REST APIs use common HTTP methods like GET, POST, PUT, and DELETE.

HTTP Methods and CRUD Operations

REST APIs use **HTTP methods** to perform different actions, which map to basic data operations (**CRUD**):

- **POST** → **(C)reate**: Add new data (e.g., create a new user).
- **GET** → **(R)ead**: Fetch data (e.g., view a list of users).
- **PUT** → **(U)pdate**: Modify existing data (e.g., update a user's details).
- **DELETE** → **(D)elele**: Remove data (e.g., delete a user).

Example: A student database

- A **POST** request to add a new student
- A **GET** request to get the name of a specific student
- A **PUT** request to update the name of a student
- A **DELETE** request to remove a student from the database

Client-Server Interaction (Request + Response)

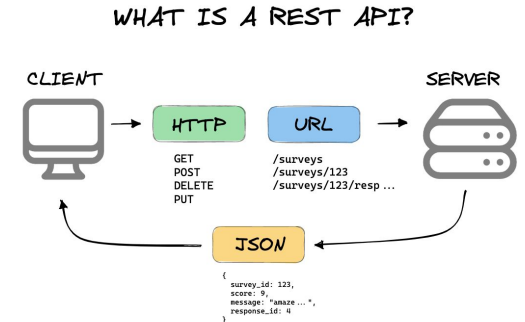
The client sends a **request** to the server, and the server sends back a **response**.

Request:

- Contains:
 - **HTTP Method** (e.g., GET, POST)
 - **URL** (communicates the resource being accessed (eg: '/student' means that we are accessing the student resource))
 - **Headers** (extra info, like authentication)
 - **Body** (data (usually in JSON format) containing required fields)

Response:

- The server replies with:
 - **Status Code** (e.g., 200 OK, 404 Not Found)
 - **Headers** (metadata like content type)
 - **Body** (the actual data, usually in JSON format)



Common HTTP Status Codes

200 OK: The request was successful.

201 OK Created: A new resource was created.

400 BAD Request: There's something wrong with the request (e.g., missing data).

404 Not Found: The requested resource does not exist.

500 Internal Server Error: Something went wrong on the server.

Use these codes to properly communicate what went wrong

Rest Apis - C++ and Crow

Crow is a C++ framework that makes it easy to build REST APIs.

Why Crow?

- Simple and lightweight
- Supports standard HTTP methods like GET, POST, PUT, and DELETE for CRUD operations

Example Data

For this demo we can take a simple local “database” that stores student information

Each student has the following fields:

- A unique integer id
- Their name

C++ Crow Demo

Basics of C++ and Crow routes

Download crow from <https://crowcpp.org/master/> and add the include directory into your CMakeLists.txt. You can then include crow functionalities with `#include "crow.h"`

```
// Crow app
```

```
crow::SimpleApp app;
```

Initialize crow app

Route name

HTTP method

```
CROW_ROUTE(app, "/students").methods(crow::HTTPMethod::POST)  
( [&students](const crow::request& req){
```

Lambda for local variables

Method parameters

Resources

Crow instructions + documentation: <https://crowcpp.org/master/guides/app/>

MongoDB with C++: <https://www.mongodb.com/docs/languages/cpp/>

Curl documentation: <https://curl.se/docs/>

Google cloud API: <https://cloud.google.com/cpp/docs/reference>

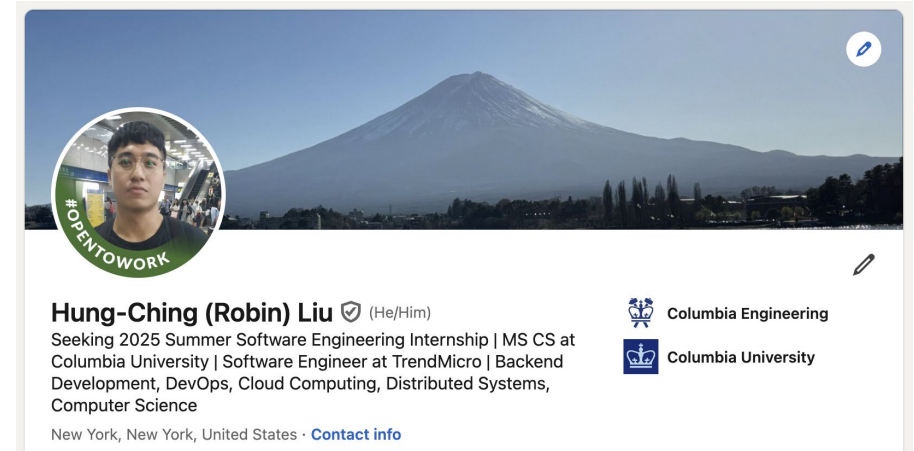
Resource being updated

- Student id
- Name

/students/{ID}

Self Introduction: Robin Liu

- From Taiwan
- 1st year in MS CS
- 2 years industry experience of backend, data ETL, cloud devops
- LinkedIn
 - www.linkedin.com/in/hung-ching-liu



Contents

- Java Spring Boot
- OpenAPI and swagger documentation
- API clients: tools for RESTful Endpoints testing

Java Spring Boot

- Commonly used Java framework for writing web services
 - Take care of the tedious part of the code, you only have to focus on your business/app logic
- Dependency Injection
- Convention over configuration (COC)
- Spring initializr
- Easy and fast to develop
 - A 15-minute tutorial on coding a RESTful web application.
 - <https://spring.io/guides/gs/rest-service>



Spring Boot Annotations

Class Level

- `@RestController`

Method Level

- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`

Param Level

- `@PathVariable`
- `@RequestParam`
- `@RequestBody`

```
12  @RestController  Hung-Ching Liu *
13  public class StudentController {
14
15      private final StudentService studentService; 6 usages
16
17      public StudentController(StudentService studentService) {  Hung-Ching Liu
18          this.studentService = studentService;
19      }
20
21      @GetMapping("/")  Hung-Ching Liu
22      public ResponseEntity<?> getIndex() {
23          return ResponseEntity.ok( body: "Hello World");
24      }
25
26      @GetMapping("/students/{id}") new *
27      public ResponseEntity<?> getStudent(@PathVariable int id) {
28          Student student = studentService.getStudentById(id);
29          return ResponseEntity.ok(student);
30      }
```

Open API and Swagger file

- OpenAPI Specification (OAS)
- Standard for HTTP APIs

```
26 dependencies { Edit Starters...  
27     implementation 'org.springframework.boot:spring-boot-starter-web'  
28     // https://mvnrepository.com/artifact/org.springdoc/springdoc-openapi-starter-webmvc-ui  
29     implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.6.0'  
30 }
```

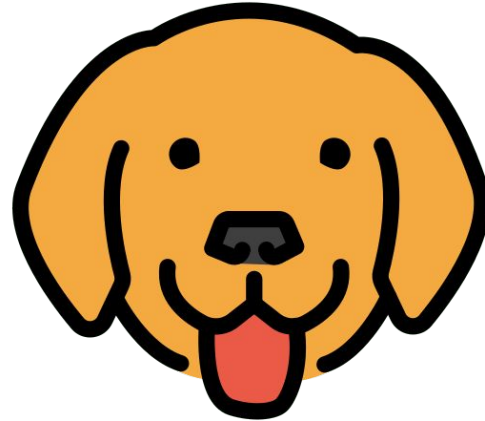
The image shows the Swagger UI interface for an OpenAPI definition. At the top, the Swagger logo is visible next to the text "powered by SMART BEAR". A search bar contains the text "/v3/api-docs" and an "Explore" button is to its right. Below this, the text "OpenAPI definition" is displayed, followed by "v0" and "OAS 3.0" in a green box. Underneath, the URL "v3/api-docs" is shown. A "Servers" section contains a dropdown menu with the selected value "http://localhost:8080 - Generated server url". The main content area is titled "student-controller" and lists five API endpoints, each with a colored button indicating the HTTP method and a dropdown arrow to the right:

- GET** /students/{id} (blue button)
- PUT** /students/{id} (orange button)
- DELETE** /students/{id} (red button)
- POST** /students/ (green button)
- GET** / (blue button)

Below the endpoints, a "Schemas" section is visible, containing a single entry "Student" with a right-pointing arrow.

RESTful endpoints testing

- API clients
 - Postman
 - Bruno



Demo(Java)

- Spring Boot
- Swagger demo
- RESTful Endpoint testing with Brunos

Resources

Spring Boot: <https://spring.io/projects/spring-boot>

OpenAPI standard: <https://www.openapis.org/>

Spring openapi plugin:

<https://github.com/springdoc/springdoc-openapi-gradle-plugin>

Postman:

<https://www.postman.com/>

Bruno:

<https://www.usebruno.com/>